

Dev. efficace

R3.02

Stéphane Mérillou - IUT du Limousin

|

Présentation générale

I- Savoirs de références

- Développement de structures de données complexes (par exemple les arbres cette année ...)
- Premières approches de l'analyse de la performance
- **Choix du langage : C++**
 - Principe des langages compilés
 - Performance :
 - *Ses bonnes performances, et sa compatibilité avec le C en font un des langages de programmation les plus utilisés dans les applications où la performance est critique. (source Wikipedia)*
- **On se basera systématiquement sur l'algo de BUT I**

Rappels : éléments caractéristiques d'un algo / prog

- un glossaire spécifiant les "données" utilisées par l'algorithme
- un ensemble d'instructions élémentaires :
- un ordre sur l'exécution

=> Valide en C++

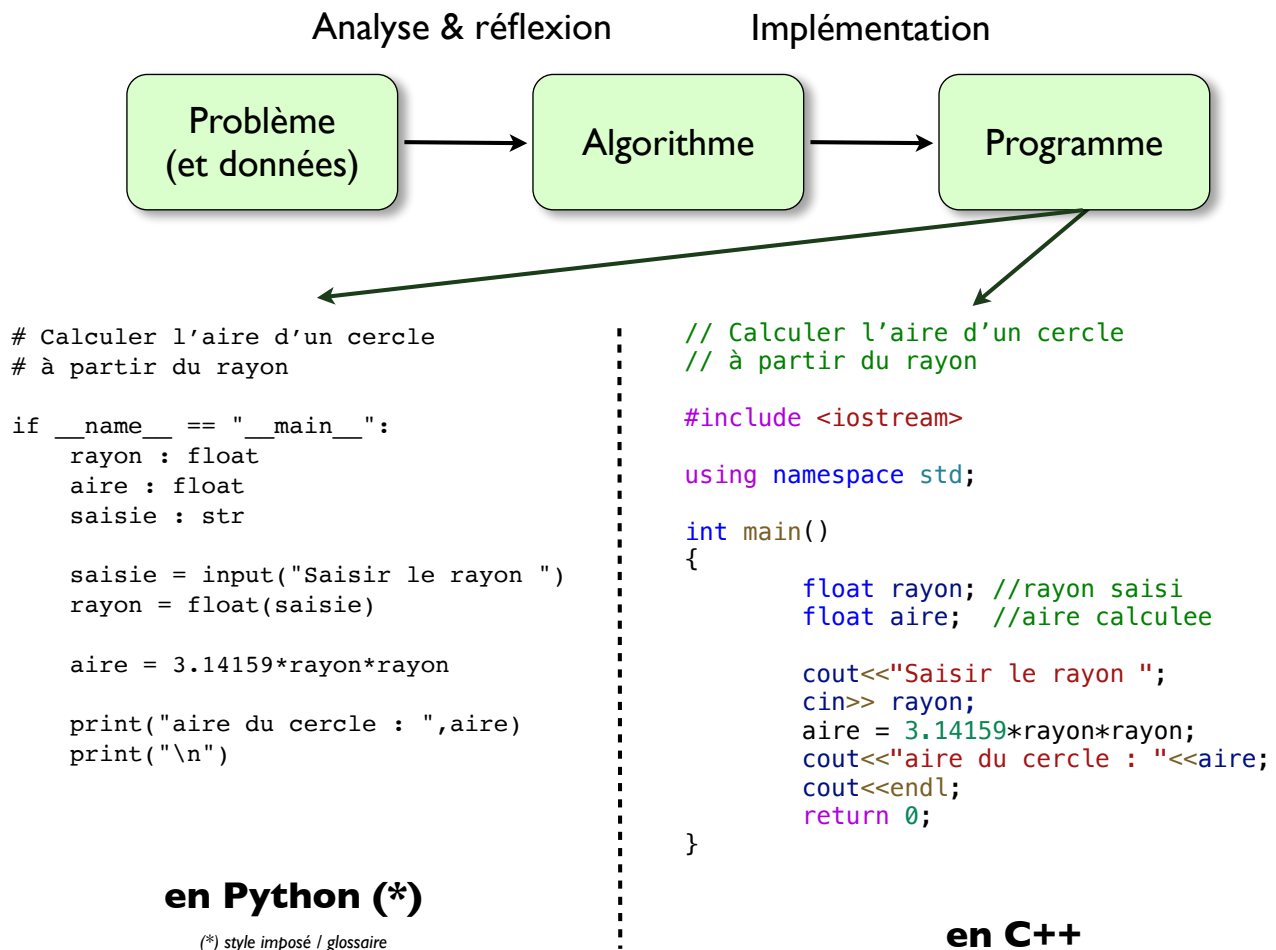
Remarques C / C++ :

- C++ est un sur-ensemble de C,
- C++ ajoute en particulier l'orienté-objet (classes, héritage, polymorphisme)

C++ : langage complexe qui évolue très régulièrement : dernière version stable C++20 et C++23 (tout n'est pas encore à jour dans les compilateurs / librairies) - la suivante est déjà en cours d'élaboration : C++26

Nous n'allons en voir que la base ...

3



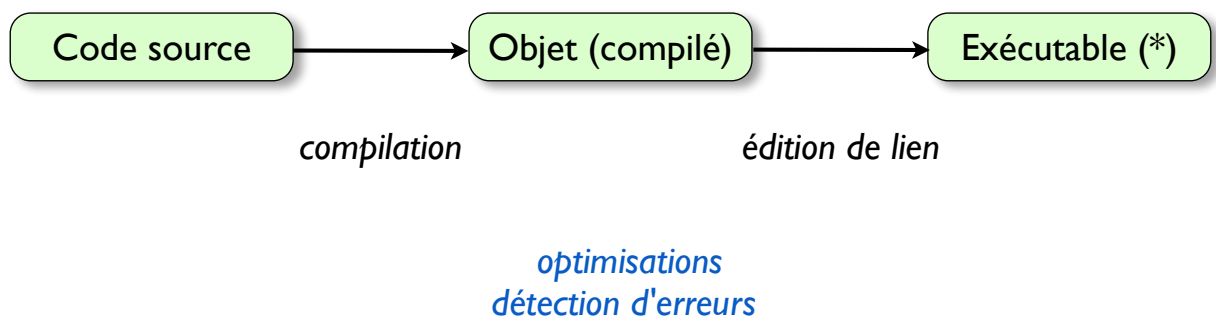
4

Langages de programmation

Il existe différents types de langages de programmation :

- interprétés : l'exécution se fait au cours de la lecture;
- **compilés : la traduction est faite une fois pour toutes en langage machine - les exécutions sont plus rapides.**

Trois états pour les langages compilés :



5

Structure générale d'un programme C/C++

```
// fichier principal (.cpp en C++ et .c en C)
// ne pas oublier les commentaires en tête

[ CLAUSES ]    // inclusions (modules externes ...)

void main() // ici commence le programme
{
    [ DÉCLARATIONS ]    // variables

    [ CORPS ]           // commandes C/C++
}
```

6

Exemple :

```
#include <iostream> // module permettant d'utiliser cout et cin
using namespace std;

int main() // ici commence le programme
{
    float nb_1, nb_2, max; // déclarations

    // saisie des nombres à comparer
    cout<<"Nombre Nr. 1 : ";
    cin>>nb_1;
    cout<<"Nombre Nr. 2 : ";
    cin>>nb_2;
    // effectuer la comparaison et calculer max
    if (nb_1 > nb_2)
    {
        max = nb_1; // affectation
    }
    else
    {
        max = nb_2;
    }
    // afficher le résultat final
    cout<<"Le maximum vaut : "<<max;
    // aller à la ligne
    cout<<endl;
    return 0;
}
```

g++ -Wall maximum.cpp -o maximum

7

Types et opérations de base en C/C++

Tous les types de base développés et présentés en algorithmique sont disponibles en C++

Un programme C/C++ est composé d'une fonction **principale** : `int main()` qui renvoie un entier au système (le `return 0;` de l'exemple)

Des accolades sont utilisées pour marquer le début et la fin d'un block.

Il est important de **soigner la présentation**, en gérant l'**indentation**, qui n'est pas obligatoire dans la syntaxe

Les instructions sont clôturées par un **point-virgule**

Les clauses en C/C++

Les clauses `#include` permettent d'inclure des modules externes qui contiennent des fonctionnalités non fournies par le programme lui-même.

Par exemple, `#include <iostream>` contient la définition des opérations de saisie et d'affichage (`cout<<` et `cin>>`).

Pour pouvoir utiliser des fonctions mathématiques, il faut inclure un module mathématique par `#include <cmath>`

Le type booléen

Le type booléen peut avoir 2 états : vrai ou faux. Les mots clefs utilisés sont :

- `bool` // pour la déclaration
- `true, false` // pour vrai, faux

9

En C/C++, les opérations non, et, ou sont codées respectivement à l'aide des symboles `!`, `&&` et `||`

Par exemple :

```
bool a, b, c ;

a = true;      //initialisations
b = false;

c = ! a;       // non a
c = a && b;     // a et b
c = a || b;    // a ou b
```

Le type entier

Le type entier utilise le mot clé `int` (abréviation de l'anglais integer, qui signifie entier). Il existe cependant plusieurs types d'entiers, dépendant de la précision voulue.

- `int short valeur; // entier signé court`
`// codé sur 2 octets`
- `int valeur; // un entier signé long`
`// codé sur 4 octets`
- `unsigned int short valeur; // entier non signé`
`// court (2 octets)`
- `unsigned int valeur; // un entier non signé`
`// long (4 octets)`

||

Les bornes de ces différents types d'entier dépendent entre autres de la machine et du compilateurs. Le compilateur dispose donc de ces constantes. **Attention, en cas de dépassement les calculs deviennent faux !**

Par exemple, actuellement :

- un entier court signé est compris dans $[-2^{15}; 2^{15}-1]$.
- un entier long signé est compris dans $[-2^{31}; 2^{31}-1]$.
- un entier court non signé est compris dans $[0; 2^{16}-1]$.
- un entier long non signé est compris dans $[0; 2^{32}-1]$.

Les opérations sur les entiers sont les suivantes : `+`, `-`, `*`, `/`, `%` (pour modulo).

Attention à la division par zéro et à **ne pas mélanger les types**.

Le type réel

Le type réel utilise les mots clé “float” ou “double” selon la précision voulue.

```
float    un_reel;    // précision simple (4 octets)
double   un_reel;    // précision double (8 octets)
```

Les opérations usuelles sont: +, -, *, /

Les fonctions mathématiques, accessibles à condition d’inclure dans les clauses `#include <cmath>` ou `#include <math.h>`

Exemples `cos`, `sin`, `exp`, `log`, `sqrt` et `pow`. Ces opérations s’appliquent à des réels en précision double.

Pour des réels en précision simple, il faut faire précéder de 'f' le nom des fonctions: `fcos`, `fsin`, `fexp`, etc.

13

Attention à ne pas mélanger les types y compris numériques !

```
nombre = 4;    // FAUX si nombre est un reel
```

Il faut écrire:

```
nombre = 4.0;    // OK si nombre est un double
```

Si on a besoin de "mélanger les types", on le demande **explicitement** :

```
double somme_notes, moyenne;
int nb_notes;
moyenne = somme_notes / double(nb_notes);
```

Attention, les conversions ne sont pas toutes possibles. Une conversion peut également impliquer une **perte de précision** sévère : en convertissant un réel en un entier, on perd la partie fractionnaire (5.978 devient 5 !).

14

Le type caractère

Le type caractère utilise le mot clef `char` (abréviation de `character`). Il est codé sur 1 octet.

Les constantes sont écrites entre `' '`, par exemple `'a'`, `'b'`, `'A'`, `'+'`, etc.

Le type chaîne de caractères

Déclaration (bibliothèque à inclure : `#include <string>`) :

```
string    une_chaine;    // chaîne de caractères
```

Opérations :

```
une_chaine = "ceci est une chaîne";
ma_chaine = "Bonjour" + " " + "Monsieur";
ma_chaine = ch1 + "suite";
if (chaine1 <= chaine2) ...
```

Les autres opérations seront détaillées en TP lorsque nécessaire (en particulier longueur et partie de chaîne).

15

Les flux en C++

il faut inclure le module `<iostream>`

Pour afficher avec les "flux", on utilise `cout` et le symbole `<<` pour séparer les éléments à afficher:

```
cout << "Le prix est " << un_entier << "euros";
```

De même la saisie utilise `cin` et le symbole `>>` :

```
cin >> un_entier;
```

Ces opérations vérifient les types (et sont plus simples d'utilisation que les correspondantes en C).

16

Structures de contrôle en C/C++

Branchements conditionnels

```
if (<cond>)  
{  
    TRAITEMENT_1 ;  
}  
else  
{  
    TRAITEMENT_2 ;  
}
```

- attention aux **parenthèses** pour la condition
- ne pas oublier **les accolades**

17

Les sélections multiples se traduisent par:

```
switch <variable>  
{  
    case 1 : Action1 ;  
            break ;  
    case 2 : Action2 ;  
            break ;  
    ...  
    default : Action_D ;  
}
```

En C++, lorsqu'une condition est vérifiée, tous les traitements qui suivent sont exécutés dans l'ordre (donc y compris ceux qui correspondent à des cas différents).

Pour éviter de traiter ces cas, il faut utiliser une commande "break" à la fin de chaque traitement (sauf lorsque l'on veut évidemment réellement exécuter ces traitements).

18

Les boucles conditionnelles se traduisent par :

Tantque <cond> faire TRAITEMENT Finfaire	while (<cond>) { TRAITEMENT ; }
Répéter TRAITEMENT Tant que <cond>	do { TRAITEMENT ; } while(<cond>);
Pour <init,pas> faire TRAITEMENT Finfaire	for (<init>; <cond>; <inc>) { TRAITEMENT ; }

Détails des 3 paramètres du "pour" :

- l'initialisation contient l'affectation de la valeur de départ (par exemple : $i=1$).
- la condition correspond à un while, par exemple : $i \leq n$.
- enfin, l'incrément indique le pas, par exemple : $i=i+1$.

19

Exemple : afficher les nombres entiers entre 1 et n :

```
#include <iostream>
using namespace std;

int main()
{
    int n ;    // jusqu'où compter
    int i ;    // compteur pour la boucle

    cout<<"Saisir un nombre entier : " ;
    cin>>n ;

    for (i=1 ; i<=n ; i=i+1)
    {
        cout<<i<<" " ;
    }

    cout<<endl ;
    return 0;
}
```

Fonctions & procédures

La fonction de calcul proprement dite est placée dans un premier fichier nommé **factoriel.cpp**, et le programme principal dans **affichefact.cpp** :



```
// fonction qui calcule factoriel n
int factoriel(int n)
{
    int i ; //compteur
    int res ; // resultat du calcul

    res = 1 ;
    for (i=2 ; i<=n ; i=i+1)
    {
        res = res*i ;
    }
    return (res) ;
}
```



```
#include <iostream>
using namespace std;

int main()
{
    char encore ; // réponse de l'utilisateur
    int i ; // l'entier saisi
    int factoriel(int) ; // la fonction factoriel

    do
    {
        cout<<"Encore un nombre :" ;
        cin>>encore ;
        if (encore=='o')
        {
            cout<<"Entre l'entier :" ;
            cin>>i ;
            if (i>=0)
            {
                cout<<"Factoriel "<<i<<" =" <<factoriel(i);
                cout<<endl;
            }
        }
    }
    while(encore=='o') ;
    return 0;
}
```

21

- Importance de la déclaration pour que le compilateur et l'éditeur de lien puissent fonctionner :
 - Le compilateur va vérifier que la fonction est utilisée correctement
 - L'éditeur de lien cherchera à "lier" la bonne fonction à notre code
- Dans le cas des fonctions prédéfinies (exemple maths : cos, sin, exp, etc.), il n'est pas nécessaire de déclarer chaque fonction avant son utilisation (c'est le rôle des clauses #include en C par exemple).

Ainsi, lorsqu'on écrit #include <cmath> les fonctions contenues dans la bibliothèque sont déclarées automatiquement et deviennent directement utilisables.

Les passages de paramètres par valeur / par référence

Avec l'exemple de l'année dernière : un programme permettant de saisir le nom et le prénom d'une personne, puis de lui proposer de changer de prénom.

Dans un premier fichier nommé : `prenom.cpp`

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string nom, prenom;
    void SaisirPersonne(string& nom, string& prenom);
    void AfficherMessage(string nom, string prenom);
    void NouvPrenom(string nom, string& prenom);

    SaisirPersonne(nom, prenom);
    AfficherMessage(nom, prenom);
    cout<<"Changement de prenom "<<endl;
    NouvPrenom(nom, prenom);
    AfficherMessage(nom, prenom);
    return 0;
}
```

travail sur les originaux
(passage par référence)

travail sur les copies
(passage par valeur)

23

Dans un second fichier, nommé `saisie.cpp` :

```
#include <iostream>
#include <string>
using namespace std;

void SaisirPersonne(string& nom, string& prenom)
{
    cout<<"Entre ton nom : ";
    cin>>nom;
    cout<<"Entre ton prenom : ";
    cin>>prenom;
}
```

Dans un troisième fichier, nommé `affich.cpp` :

```
#include <iostream>
#include <string>
using namespace std;

void AfficherMessage(string nom, string prenom)
{
    cout<<"bonjour "<<prenom<<" "<<nom<<endl;
}
```

24

Dans un quatrième fichier, nommé `nouvprenom.cpp` :

```
#include <iostream>
#include <string>
using namespace std;

void NouvPrenom(string nom, string& prenom)
{
    void AfficherMessage(string nom, string prenom);

    AfficherMessage(nom, prenom);
    cout<<"Entre ton nouveau prenom : ";
    cin>>prenom;
}
```

25

Modules en C++

Un **module** correspond à deux fichiers séparés:

- l'un contient la spécification (fichier **.h**, ou **.hpp** ou ...),
- l'autre l'implémentation (fichier **.cpp** ou ...)

Les fichiers correspondant à un module sont compilés indépendamment du programme les utilisant.

Exemple :

```
#ifndef OUTILS_H          // tres important !
#define OUTILS_H          // Le nom OUTILS_H est choisi par le programmeur

#include <string>

using namespace std;

// PPC ici à détailler
void SaisirPersonne(string& nom, string& prenom);

// PPC ici à détailler
void AfficherMessage(string nom, string prenom);

// PPC ici à détailler
void NouvPrenom(string nom, string& prenom);

#endif
```

Les **#ifndef**, **#define** et **#endif** permettent de n'inclure qu'une seule fois le « **.h** » dans le programme !

26

L'implémentation est placée dans un fichier portant l'extension habituelle **".cpp"** pour le C++ :

```
#include <iostream>
#include "outils.h"

void SaisirPersonne(string& nom, string& prenom)
{
    cout<<"Entre ton nom : ";
    cin>>nom;
    cout<<"Entre ton prenom : ";
    cin>>prenom;
}

void AfficherMessage(string nom, string prenom)
{
    cout<<"bonjour "<<prenom<<" "<<nom<<endl;
}

void NouvPrenom(string nom, string& prenom)
{
    void AfficherMessage(string nom, string prenom);

    AfficherMessage(nom, prenom);
    cout<<"Entre ton nouveau prenom : ";
    cin>>prenom;
}
```

Il est important d'**inclure la spécification** dans l'implémentation du module. Ceci ce fait grâce à la clause `#include`, mais le nom est indiqué entre " " et non entre < >, car il ne s'agit pas d'un module du système, mais d'un module personnel

27

programme utilisant une liste d'entiers

```
#include <iostream>
#include <string>
#include "outils.h"

using namespace std;

int main()
{
    string nom, prenom;

    SaisirPersonne(nom, prenom);
    AfficherMessage(nom, prenom);
    cout<<"Changement de prenom "<<endl;
    NouvPrenom(nom, prenom);
    AfficherMessage(nom, prenom);

    return 0;
}
```

Grâce à la spécification, il n'est plus nécessaire de déclarer les procédures et fonctions avant leur utilisation, ceci étant fait dans le fichier spécification inclus en tête de programme.

28

Les tableaux

```
int    vecteur[10];
float  tab[100];
char   nom[50] ;
```

L'accès aux cases du tableau se fait par le biais des crochets (**première case d'indice 0**):

```
x = vecteur[5];           // place le 6ème élément dans x
y = vecteur[i+2];
```

Lors de l'exécution, **les indices ne sont pas vérifiés**. En cas de débordement, le programme passe dans un état instable, qui peut dans le meilleur des cas conduire à un arrêt immédiat, ou dans le pire, à la poursuite avec des valeurs erronées.

```
x = vecteur[10];
y = vecteur[-1];
```

29

Programme : remplir un tableau de 10 valeurs aléatoires entre 0 et 100, puis les afficher.

```
#include <iostream>
#include <cstdlib>      // pour utiliser rand()
#include <ctime>         // pour utiliser time()

using namespace std;

const int TAILLE=10;

int main() {
    int tab[TAILLE], i ; // OK car TAILLE est une constance !

    srand(time(NULL));   //sranddev();

    for (i=0; i<TAILLE; i=i+1)
    {
        // rand retourne une valeur entière entre 0 et RAND_MAX
        tab[i]=(int)(((float)rand()/(float)RAND_MAX)*100.0);
    }

    for (i=0; i<TAILLE; i=i+1)
    {
        cout<<i<<" "<<tab[i]<<endl;
    }
}
```

30

Tableaux à N-dimensions

On peut déclarer en C/C++ des tableaux de plus d'une seule dimension :

```
float matrice[50][50]; // indices vont de 0 à 49 !
```

Les accès restent les mêmes, sauf qu'il faut indiquer deux dimensions (deux indices).

```
matrice[0][0] = 3.14159 ;  
matrice[50][1] = 5.0 ; // erreur : indice 50 n'existe pas
```

Exemple saisir une matrice carrée de taille TAILLE :

```
for (i=0 ; i<TAILLE ; i=i+1) {  
    for (j=0 ; j<TAILLE ; j=j+1) {  
        cout<<"Saisir ("<<i+1<<","<<j+1<<") : " ;  
        cin>>matrice[i][j] ;  
    }  
}
```

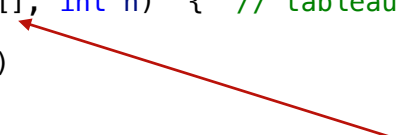
Bien évidemment, on peut encore augmenter le nombre de dimensions (pour 3, on utilisera 3 crochets, pour 4, 4 crochets, etc.).

En C++ les tableaux sont toujours passés par **référence** (ou par adresse) sans besoin de le préciser

31

Exemple : programme qui place dans un tableau les carrés des entiers de 1 à 10

```
#include <iostream>  
using namespace std;  
  
void aff_tab(const int t[], int n) { // tableau en entrée  
    int i;  
    for (i=0; i<n; i++)  
        cout<<t[i]<<" ";  
    // t[0]=100; interdit ! erreur de compilation (const)  
    cout<<endl;  
}  
  
void remplir_tab(int t[], int n) { // tableau en sortie  
    int i;  
    for (i=0; i<n; i++)  
        t[i] = i*i;  
}  
  
int main() {  
    int tab[10], n;  
  
    n = 10;  
    remplir_tab(tab, n); // tab en sortie  
    aff_tab(tab, n);    // tab en entrée  
}
```



Taille 1ère dim facultative

32

Structures de données / Classes

I. Le type structure ou enregistrement

Il arrive très fréquemment de travailler en informatique avec des données "complexes" ou "composées" (images, sons, clients, factures, adresses, etc.)

Toutes ces informations peuvent se composer à partir des types de base.

Par exemple, un *client* est identifié par :

un nom	chaîne de caractères
un prénom	chaîne de caractères
un numéro de téléphone	chaîne de caractères

Pour faciliter la gestion de ces données complexes et, afin d'éviter une quantité trop importante de variables, il est possible de regrouper les informations dans un nouveau type consistant en un assemblage de types de base.

33

Structures en C/C++

Les structures se définissent de la manière suivante en C/C++ :

```
typedef struct {  
    string  nom;  
    string  prenom;  
    string  tel;  
} t_personne ;  
  
t_personne lapers ;
```

Pour accéder aux membres, on utilise l'opérateur . :

```
cin>>lapers.nom ;
```

Le regroupement d'une structure et des fonctions associées à cette structure compose une classe, qui permet de créer des objets.

34

Classes & objets en C++

Classe :

- Regroupement de données (attributs ou champs) et de méthodes (fonctions membres)
- Extension des structures (struct) avec différents niveaux de visibilité (protected, private et public)

Objet : instance de classe

- Valeurs des attributs propres à chaque objet
- Attributs et méthodes communs à tous les objets d'une classe

Encapsulation

- Caractérisation d'un objet par les spécifications de ses méthodes : interface
- Indépendance vis à vis de l'implémentation

35

Classes & objets en C++

```
#include <string>
using namespace std;

class Personne
{
    private :
        string nom;
        string prenom;
        string tel;

    public :
        Personne();
        void SetPersonne(string, string, string);
        void Affiche();
        string getNom();
};
```

Membres privés, visibles uniquement depuis l'objet

Membres publics, utilisables à l'extérieur de l'objet

Interface de la classe

fichier personne.h

36

Classes & objets en C++

```
#include <iostream>
#include <string>
#include "personne.h"
```

```
using namespace std;
```

```
Personne::Personne() {
    nom = "";
    prenom = "";
    tel = "";
}
```

```
void Personne::SetPersonne(string n, string p, string t) {
    nom = n;
    prenom = p;
    tel = t;
}
```

```
void Personne::Affiche() {
    cout<<"Nom      : "<<nom<<endl;
    cout<<"Prénom  : "<<nom<<endl;
    cout<<"tél     : "<<nom<<endl;
}
```

```
string Personne::getNom() {
    return nom;
}
```

Constructeur ...

Donc penser aussi au **Destructeur**

fichier personne.cpp

37

Classes & objets en C++

```
#include <iostream>
#include <string>
#include "personne.h"
```

```
using namespace std;
```

```
int main() {
    Personne qqun;

    //qqun = Personne();
    qqun.Affiche();
    qqun.SetPersonne("Bond", "James", "+44 007007");
    qqun.Affiche();

    cout<<qqun.getNom()<<endl;
}
```

Etre attentif aux problèmes
d'initialisation

seule possibilité pour
accéder au nom (privé)

fichier main.cpp

38

Constructeurs & destructeur

```
class Exemple
{
    private :
        int attribut;

    public :
        Exemple();
        Exemple(int);
        Exemple(const Exemple &); // constructeur par copie
        ~Exemple();
};

// Définition du constructeur par copie
Exemple::Exemple (const Exemple& ex)
{
    attribut = ex.attribut; // Recopie champ à champ
}
```

39

Fichiers :

On distingue les fichiers **binaires** et les fichiers **textes**

Pour créer un fichier :

- on l'ouvre en écriture.
- on écrit des données dans le fichier.
- on ferme le fichier.

Pour lire des données écrites dans un fichier :

- on l'ouvre en lecture.
- on lit les données en provenance du fichier.
- on ferme le fichier.

En C / C++, 2 possibilités :

- cstdio (vient du C)
- fstream (spécifique C++)

cstdio - exemple avec fichier binaire

Par l'exemple. On dispose de la structure suivante :

```
typedef struct {  char nom[30] ;
                  char prenom[30] ;
                } pers;
```

Nous allons créer deux procédures, l'une pour créer un fichier de **pers**, l'autre pour le lire.

```
void CreerFich( string nom ) {
    pers p;
    FILE *f;
    char rep;
    f = fopen(nom.c_str(), "w"); // test à envisager ici & syntaxe !
    cout<<"Encore un ajout ? : ";  cin>>rep;
    while (rep=='o') {
        cout<<"Nom de la personne :"; cin>>p.nom;
        cout<<"prénom de la personne :"; cin>>p.prenom;
        fwrite(&p, sizeof(pers), 1, f);
        cout<<"Encore une personne :"; cin>>rep;
    }
    fclose(f);
}
```

41

La lecture de ce fichier doit être conçue en respectant le type précis des données préalablement enregistrées.

```
void LireFich( string nom ) {
    pers p;
    FILE *f;
    // ouverture
    f = fopen( nom.c_str(), "r" );
    if( f==NULL ) {
        cout<<"Cannot open file for reading data " ;
        return ;
    }
    // boucle de lecture
    fread(&p, sizeof(pers), 1, f) ;
    while ( !feof(f)) {
        cout<<"nom : "<<p.nom<<" - prenom : "<<p.prenom<<endl;
        fread(&p, sizeof(pers), 1, f) ;
    }
    // fermeture
    fclose(f);
}
```

Pour les fichiers texte : voir `fprintf` et `fscanf` ...

42

fstream - exemple avec fichier texte

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main()
{
    string ch1;
    string ch2;

    cin>>ch1; cin>>ch2;
    ofstream f ("fich.txt");
    if (!f.is_open())
        cout << "Impossible d'ouvrir le fichier en écriture !" << endl;
    else
    {
        f << "1: " << ch1 <<endl;
        f << "2: " << ch2 <<endl;
    }
    f.close();
    return 0;
}
```

Lecture avec `ifstream` et `std::getline()` ou connaissance précise du fichier

43

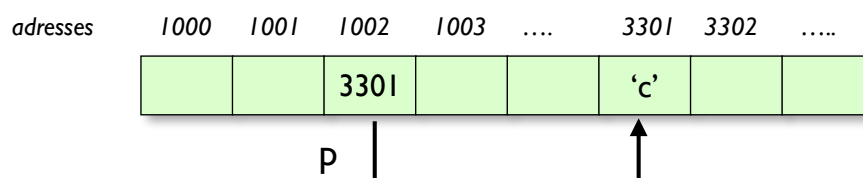
Pointeurs

I- Définition

Au cours des premiers chapitres, nous avons étudié la définition des variables. Chaque variable est caractérisée par :

- un type (entier, réel, caractère, ...);
- une valeur;
- un nom;
- une zone mémoire.

Une variable est un tiroir dans lequel, il est possible de ranger une certaine valeur. Un pointeur est une variable un peu particulière car le contenu est une adresse mémoire (par exemple l'adresse d'une autre variable).



Sur le schéma, p est un pointeur dont le contenu est une adresse. Il s'agit d'un pointeur de caractère (sur un type caractère).

44

Ici, un pointeur est nécessairement typé : il pointe vers une zone contenant un type précis (entier, réel, booléen, etc.).

```
<type> * ptr ; // La variable ptr est un pointeur de <type>.
```

Opérations sur les pointeurs (nouveau, adresse, contenu)

```
#include <iostream>
using namespace std;

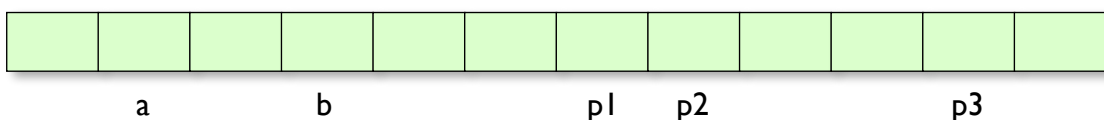
int main() {
    int a, b;
    int* p1;
    int* p2;
    int* p3;

    a = 2;
    b = 5;
    cout<<"a = "<<a<<"    b = "<<b<<endl;
    p1 = &a;           // opérateur d'adresse
    p2 = new int;       // opérateur nouveau
    *p1 = b;           // opérateur contenu
    *p2 = 3;
    // *p3 = *p2;
    p3 = p1;
    *p3 = 10;
    cout<<"a = "<<a<<"    b = "<<b<<endl;
    return 0;
}
```

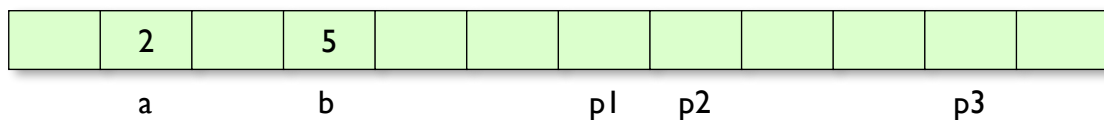
Dessiner ce qui se passe ...

45

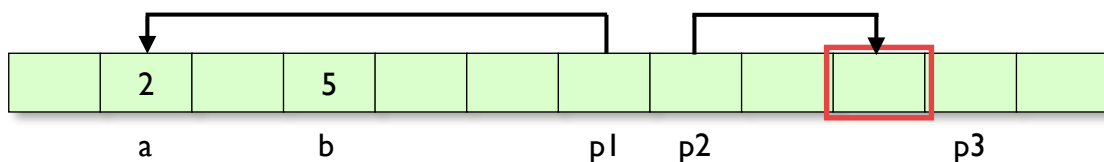
```
int a, b;
int* p1;
int* p2;
int* p3;
```



```
a = 2;
b = 5;
```



```
p1 = &a;
p2 = new int;
```

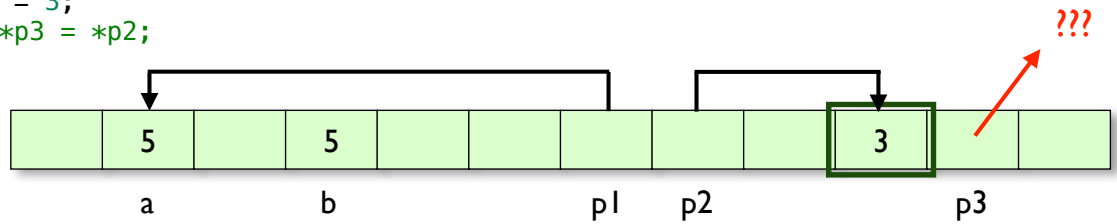


46

```

*p1 = b;
*p2 = 3;
// *p3 = *p2;

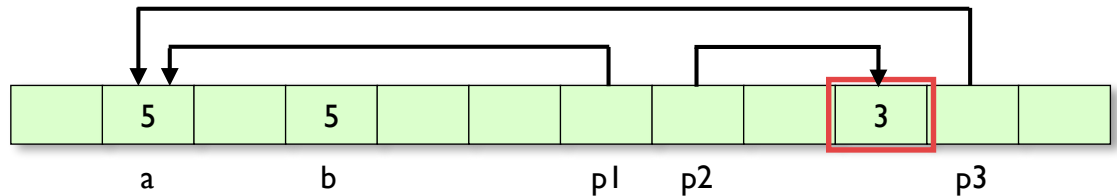
```



```

p3 = p1

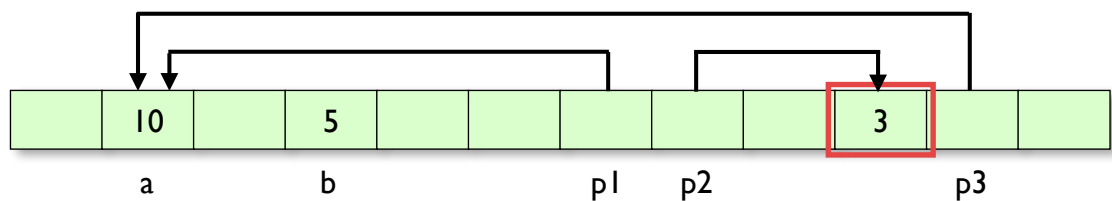
```



```

*p3 = 10

```



47

L'opérateur adresse

En C, l'opérateur adresse est codé par le symbole du « Et commercial » & :

```

int *ptr ;
int x ;
ptr = &x ; // & correspond à l'opérateur adresse

```

À présent la variable **ptr** contient l'**adresse de la variable x**.

Attention, les types de **ptr** et **x** doivent coïncider.

L'opérateur nouveau

C'est la commande **new** qui permet de faire de l'allocation dynamique :

```

int    *ptr ;
...
ptr = new int ;           // un seul entier
ptr = new int [TAILLE]   // plusieurs entiers (même déclaration)

```

Le complémentaire de **new** est **delete** : **delete ptr;** **TRES IMPORTANT !!**

48

L'opérateur contenu

il faut toujours s'assurer que le pointeur a bien été initialisé.

On code en C, cette opération à l'aide -là encore malheureusement- d'une *, qu'il ne faut ni confondre avec celle de la multiplication, ni celle de la déclaration d'un pointeur :

```
int *ptr ;
...
ptr = new int [10] ;
*ptr = 5 ;
*(ptr+1) = 6;    <=>    ptr[1] = 6
```

Tout tableau C/C++, correspond en fait à un pointeur constant (donc non modifiable)

49

Exemple : passage par adresse ...

```
#include <iostream>
using namespace std;
void exg(int* a, int* b)
{
    int c ;
    c = *a ;
    *a = *b ;
    *b = c ;
}
```

Le passage de paramètres à la procédure `exg()` se fait par valeur, on passe donc « une copie » des adresses de `x` et de `y`.

Dans la procédure, on travaille ensuite sur les contenus (sur les cases pointées par les copies des adresses).

```
int main() {
    int x,y ;

    x = 5 ;
    y = 6 ;
    cout<<"x="<<x<<" y="<<y<<endl ;
    exg(&x, &y) ;    // attention: fournir des adresses
    cout<<"x="<<x<<" y="<<y<<endl ;

    return 0;
}
```

50

Chaînages

I- Introduction

Les **tableaux** ont un inconvénient majeur : il faut allouer un espace suffisamment grand dès le départ, ce qui fait que souvent la taille allouée est en grande partie non utilisée.

Les langages « modernes » sont accompagnés de bibliothèques comprenant des structures de stockage de données particulières capables d'adapter leur taille. Notre objectif ici est de montrer une approche du fonctionnement interne de ces structures.

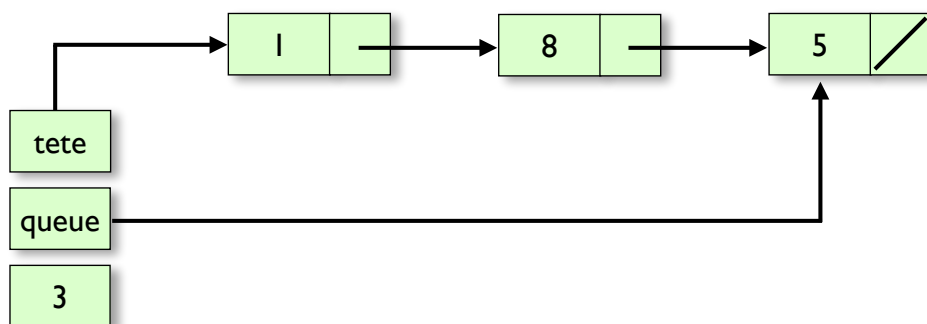
Avec les pointeurs, il est possible d'allouer un espace dynamiquement, mais, l'espace alloué ne peut pas être modifié (par exemple agrandi, si le tableau est trop petit).

Une solution peut être de créer un nouvel espace, mieux adapté en taille, de copier les données de l'ancien vers le nouveau, puis de supprimer l'ancien.

Comment peut-on être plus efficace ?

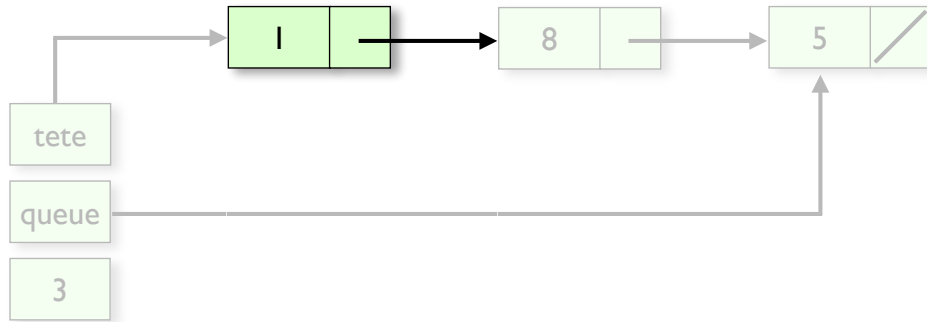
51

Chaînages de cellules en mémoire (révisions / rappels !) - exemple :



52

Chaînages de cellules en mémoire (révisions / rappels !) - exemple :



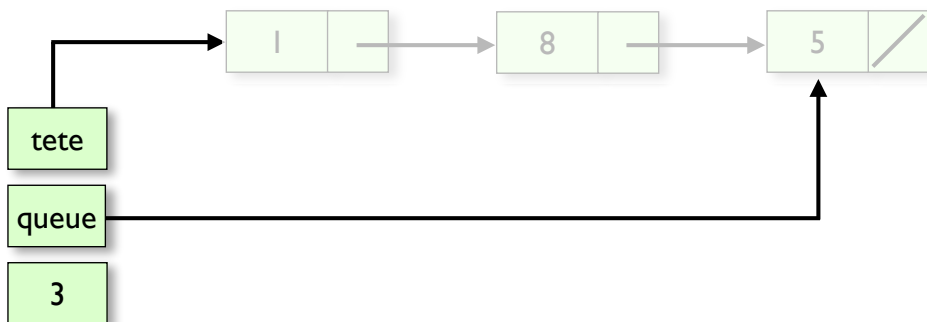
```

class maillon
{
    public:
        int v;
        maillon *suivant;

        maillon(int d) { v = d, suivant = NULL; }
        maillon(int d, maillon *nouv) { v = d, suivant = nouv; }
};
  
```

53

Chaînages de cellules en mémoire (révisions / rappels !) - exemple :



```

class chaine
{
    private:
        maillon* tete;
        maillon* queue;
        int nb_maillons;
    public:
        chaine();
        chaine(const chaine &);
        ~chaine(void);
    ...
};
  
```

54

Reste à écrire les méthodes permettant de manipuler la chaîne :

```
void adjtete();
void suptete();
void adjqueue(int valeur);
void supqueue();
int longueur() const;
int element(int position) const;
void adjpos(int valeur, int position);
void suppos(int position);
void vide(void);
bool est_vide() const;
void affiche();
```

55

Constructeurs et destructeur

```
chaine::chaine(void)
{
    // On initialise la liste.
    tete = NULL;
    queue = NULL;
    nb_maillons = 0;
}

chaine::chaine(const chaine& source)
{
    // On initialise la liste.
    tete = NULL;
    queue = NULL;
    nb_maillons = 0;

    // On copie la liste.
    maillon *nouveau = source.tete;
    while (nouveau != NULL)
    {
        adjqueue(nouveau->v);
        nouveau = nouveau->suivant;
    }
}

chaine::~chaine(void)
{
    vide();
}
```

Très important !

56

Adjonction en tête

```
void chaine::adjtete(int valeur)
{
    maillon *nouveau = new maillon(valeur);

    nouveau->suivant = tete; // nouveau pointe sur le premier.
    tete = nouveau;         // nouveau est maintenant la tete

    if (nb_maillons == 0)   // Si c'est le seul maillon,
        queue = nouveau;   // alors c'est aussi la queue.

    nb_maillons++;
}
```

< Faire un Schéma >

57

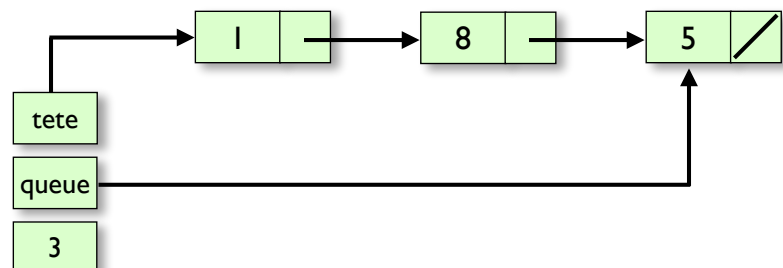
Adjonction en tête

```
void chaine::adjtete(int valeur)
{
    maillon *nouveau = new maillon(valeur);

    nouveau->suivant = tete; // nouveau pointe sur le premier.
    tete = nouveau;         // nouveau est maintenant la tete

    if (nb_maillons == 0)   // Si c'est le seul maillon,
        queue = nouveau;   // alors c'est aussi la queue.

    nb_maillons++;
}
```



58

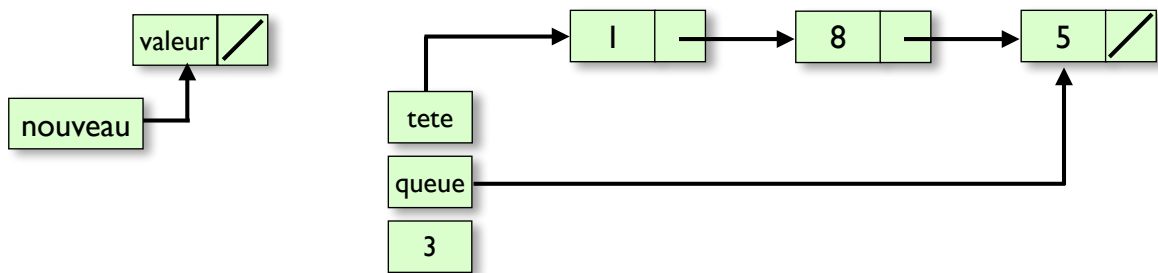
Adjonction en tête

```
void chaine::adjtete(int valeur)
{
    maillon *nouveau = new maillon(valeur);

    nouveau->suivant = tete; // nouveau pointe sur le premier.
    tete = nouveau;         // nouveau est maintenant la tete

    if (nb_maillons == 0)   // Si c'est le seul maillon,
        queue = nouveau;   // alors c'est aussi la queue.

    nb_maillons++;
}
```



59

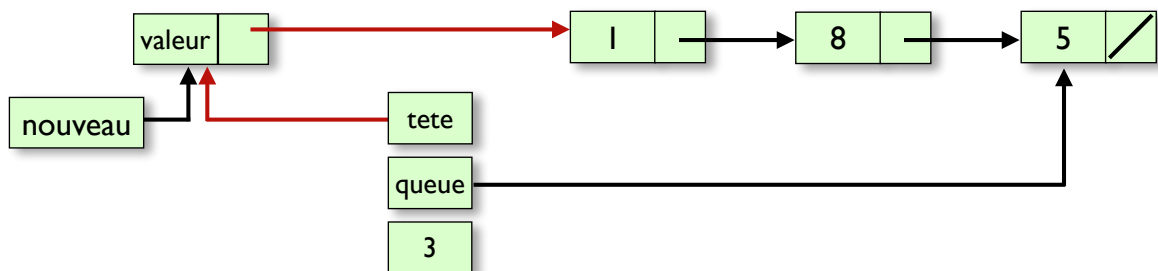
Adjonction en tête

```
void chaine::adjtete(int valeur)
{
    maillon *nouveau = new maillon(valeur);

    nouveau->suivant = tete; // nouveau pointe sur le premier.
    tete = nouveau;         // nouveau est maintenant la tete

    if (nb_maillons == 0)   // Si c'est le seul maillon,
        queue = nouveau;   // alors c'est aussi la queue.

    nb_maillons++;
}
```



60

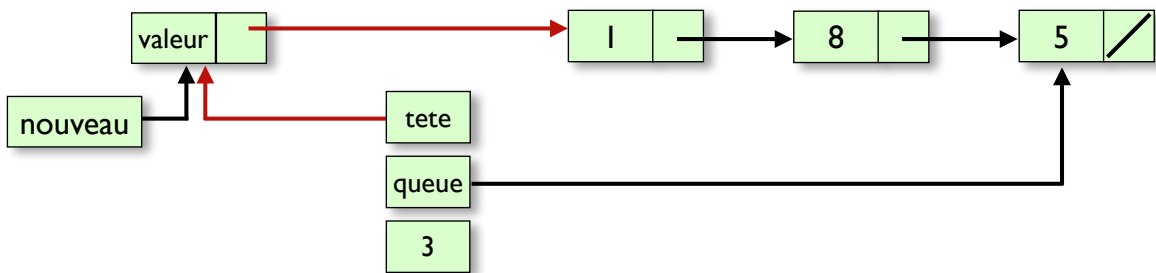
Adjonction en tête

```
void chaine::adjtete(int valeur)
{
    maillon *nouveau = new maillon(valeur);

    nouveau->suivant = tete; // nouveau pointe sur le premier.
    tete = nouveau;         // nouveau est maintenant la tete

    if (nb_maillons == 0)   // Si c'est le seul maillon,
        queue = nouveau;   // alors c'est aussi la queue.

    nb_maillons++;
}
```



61

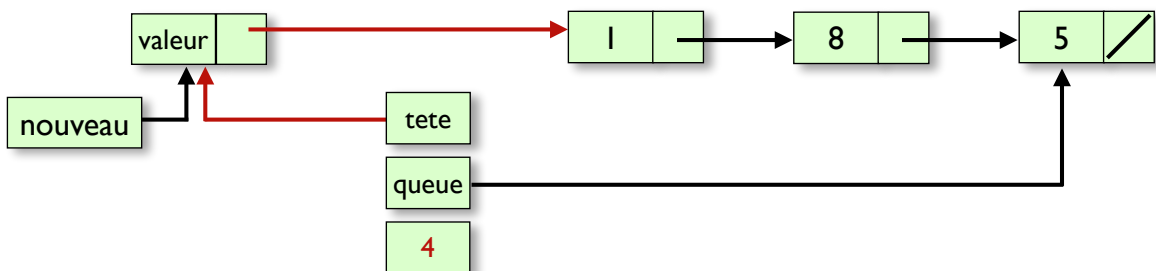
Adjonction en tête

```
void chaine::adjtete(int valeur)
{
    maillon *nouveau = new maillon(valeur);

    nouveau->suivant = tete; // nouveau pointe sur le premier.
    tete = nouveau;         // nouveau est maintenant la tete

    if (nb_maillons == 0)   // Si c'est le seul maillon,
        queue = nouveau;   // alors c'est aussi la queue.

    nb_maillons++;
}
```

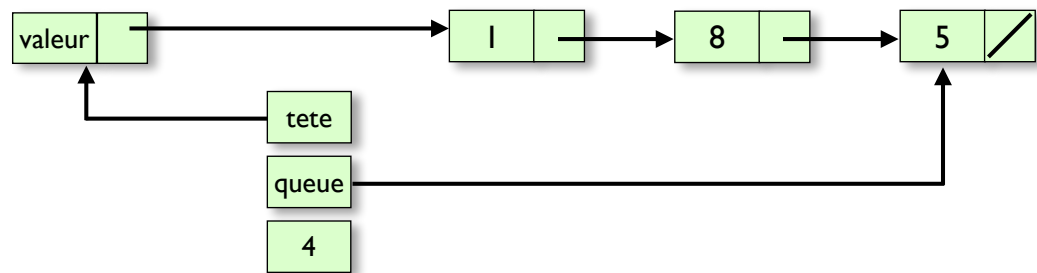


62

Suppression en tête

```
void chaine::suptete()
{
    if (nb_maillons > 0)
    {
        // On efface le maillon.
        maillon *a_effacer = tete;
        tete = tete->suisvant;
        delete a_effacer;

        // On ajuste la queue si c'était le seul maillon.
        nb_maillons--;
        if (nb_maillons == 0)
            queue = NULL;
    }
}
```

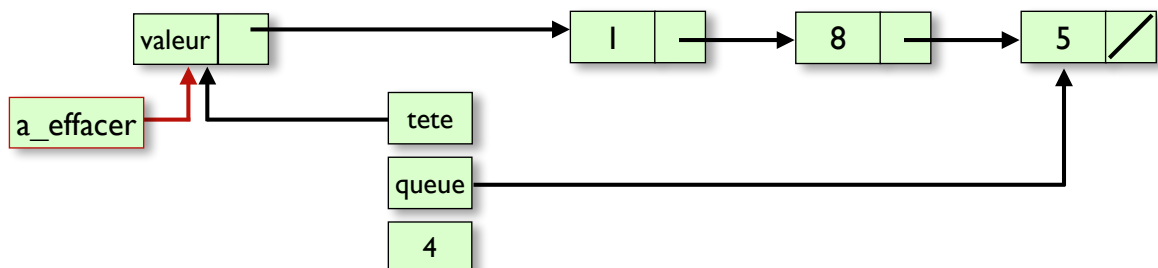


63

Suppression en tête

```
void chaine::suptete()
{
    if (nb_maillons > 0)
    {
        // On efface le maillon.
        maillon *a_effacer = tete;
        tete = tete->suisvant;
        delete a_effacer;

        // On ajuste la queue si c'était le seul maillon.
        nb_maillons--;
        if (nb_maillons == 0)
            queue = NULL;
    }
}
```

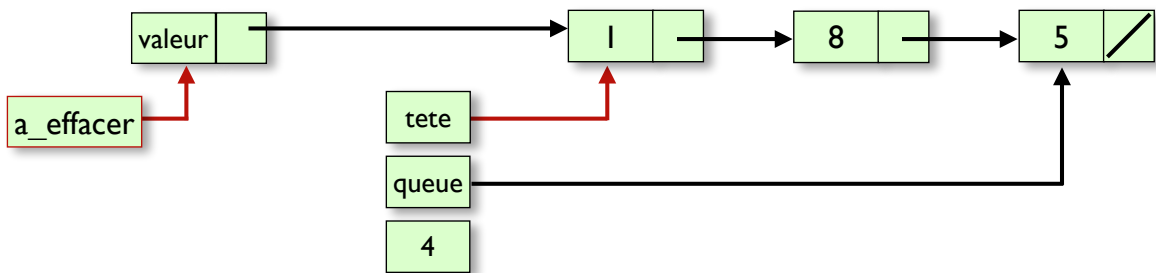


64

Suppression en tête

```
void chaine::suptete()
{
    if (nb_maillons > 0)
    {
        // On efface le maillon.
        maillon *a_effacer = tete;
        tete = tete->suisvant;
        delete a_effacer;

        // On ajuste la queue si c'était le seul maillon.
        nb_maillons--;
        if (nb_maillons == 0)
            queue = NULL;
    }
}
```

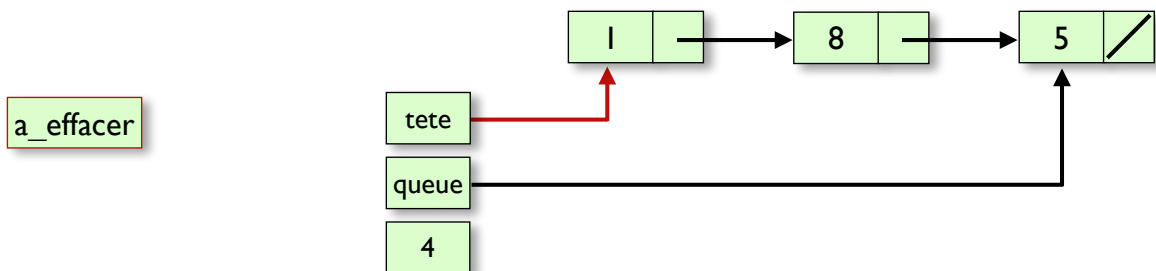


65

Suppression en tête

```
void chaine::suptete()
{
    if (nb_maillons > 0)
    {
        // On efface le maillon.
        maillon *a_effacer = tete;
        tete = tete->suisvant;
        delete a_effacer;

        // On ajuste la queue si c'était le seul maillon.
        nb_maillons--;
        if (nb_maillons == 0)
            queue = NULL;
    }
}
```

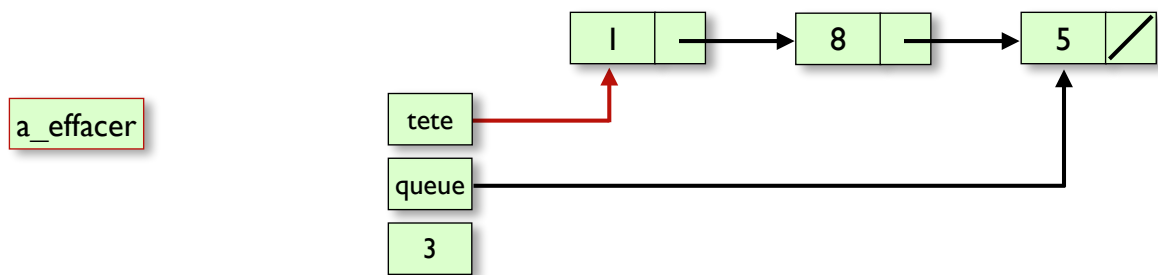


66

Suppression en tête

```
void chaine::suptete()
{
    if (nb_maillons > 0)
    {
        // On efface le maillon.
        maillon *a_effacer = tete;
        tete = tete->suivant;
        delete a_effacer;

        // On ajuste la queue si c'était le seul maillon.
        nb_maillons--;
        if (nb_maillons == 0)
            queue = NULL;
    }
}
```



67

Ajout en queue

```
void chaine::adjqueue(int valeur)
{
    if (nb_maillons == 0) // Si c'est le seul maillon,
        adjtete(valeur);
    else
    {
        maillon *nouveau = new maillon(valeur);
        queue->suivant = nouveau; // La queue pointe sur le nouveau.
        queue = nouveau;          // Le nouveau devient la queue.
        nb_maillons++;
    }
}
```

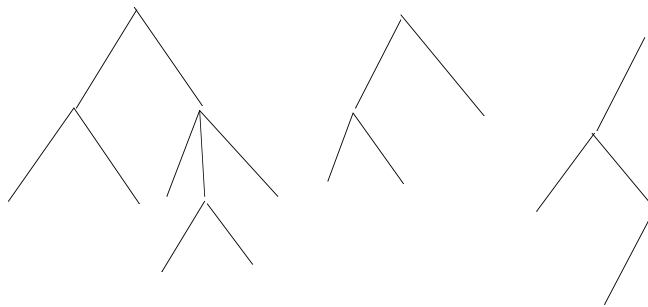
68

Arbres et Forêts

Définitions et propriétés

Une forêt sur un ensemble E est un couple (E, t) où t est une relation binaire dans E telle qu'il n'y ait pas de circuit (bouclage en suivant la relation binaire) et telle que tout noeud (élément de E) ait, au plus, un prédécesseur.

Exemples et représentations graphiques :



69

Chaque élément est un **noeud**.

On appelle **racine** un noeud sans prédécesseur.

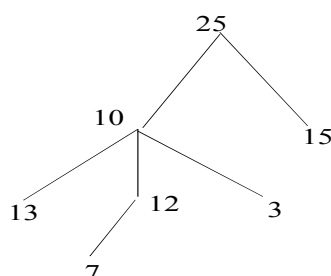
On appelle **feuille** un noeud sans successeur.

On appelle **fils** de x un élément de $t(x)$ et **père** de x l'élément $t^{-1}(x)$.

Un **arbre** est une forêt n'ayant qu'une seule racine.

Deux éléments de même père sont dits des **frères**.

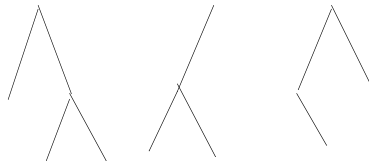
On appelle **branche** d'un arbre tout chemin de la racine à une feuille de l'arbre. On parle d'arbre **étiqueté**, lorsqu'à un noeud est associée une fonction de *valuation* :



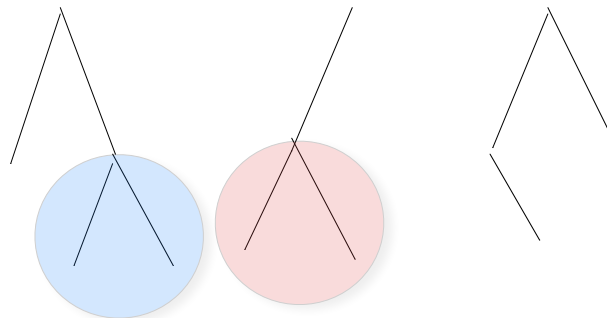
70

Arbre binaire

Un arbre binaire est un arbre tel que chaque sommet a au plus deux fils, dits **droit** et **gauche**.

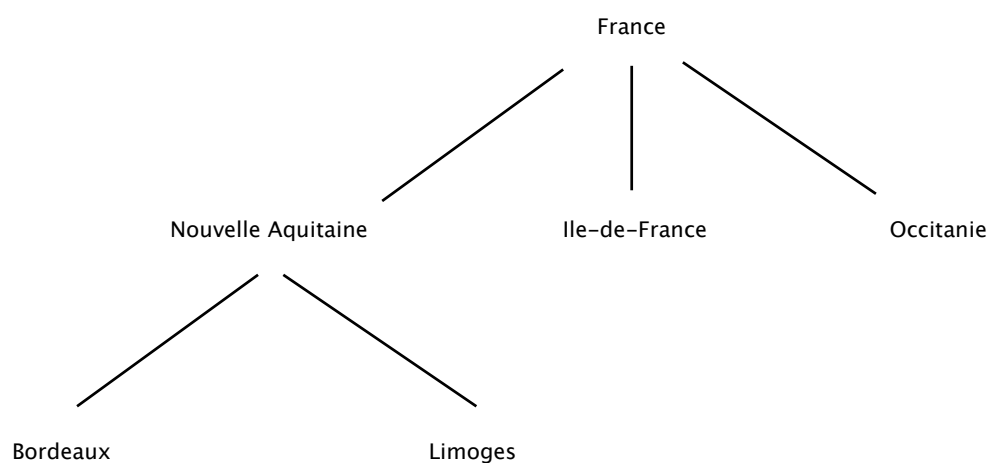


On distingue alors aussi le **sous-arbre droit** et **gauche** d'un sommet.



71

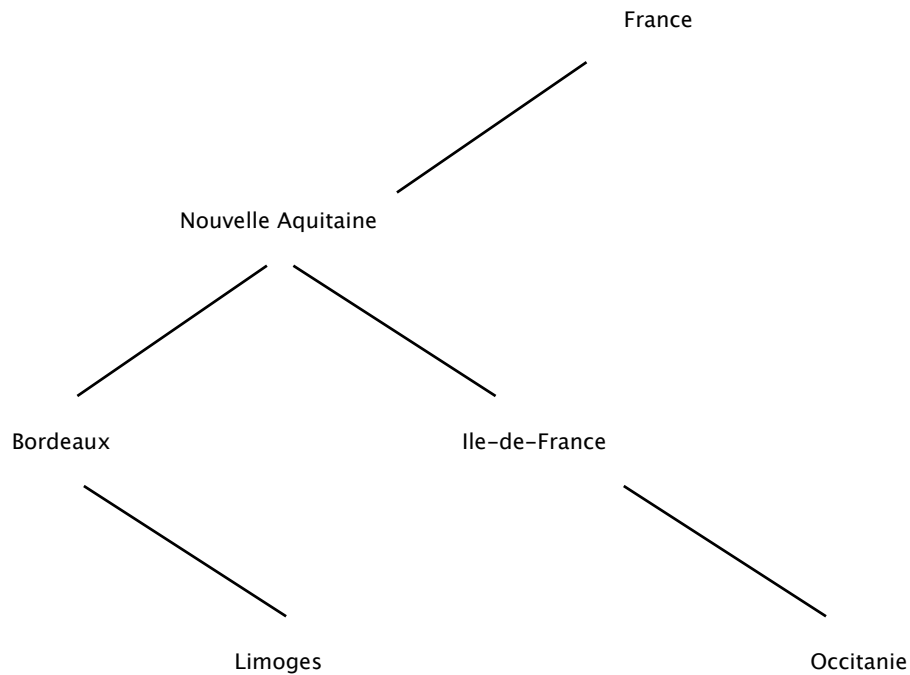
Il est possible de transformer tout arbre général en arbre binaire, tout en conservant la sémantique et sans redondance d'information (la transformation est bijective) :



Pour cela on code le fils droit et gauche ainsi : premier fils et frère ...

72

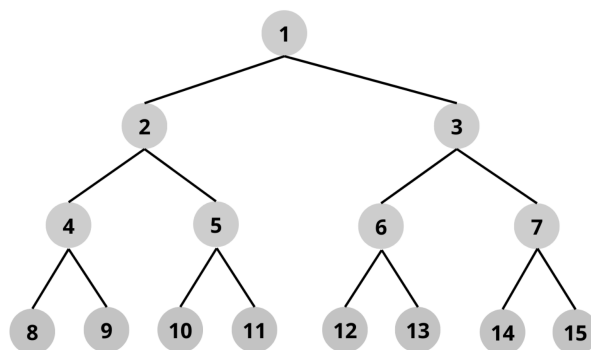
Pour cela on code le fils droit et gauche ainsi : premier fils et frère ...



73

Arbres binaires parfaits, complets

Un arbre binaire est dit **parfait (perfect binary tree)** si toutes les feuilles sont au même niveau et si tous les noeuds qui ne sont pas des feuilles ont exactement deux fils.



Un arbre binaire est dit **complet (complete binary tree)** si toutes les feuilles sont situées sur au plus deux niveaux consécutifs: l'avant dernier niveau est plein et les feuilles du dernier niveau sont groupées le plus à gauche possible

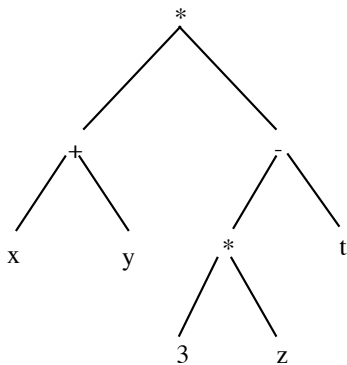
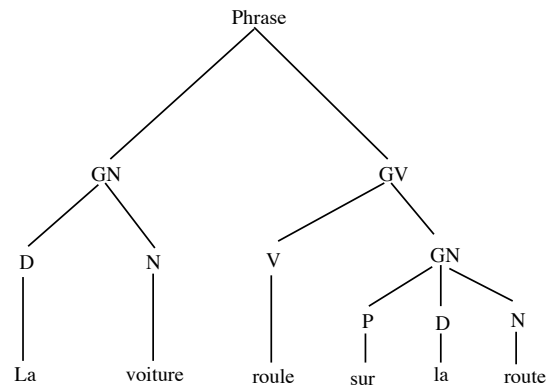
74

Utilisation des arbres

Il s'agit d'une des structures de données les plus fréquentes.

Analyse d'une phrase:

la voiture roule sur la route

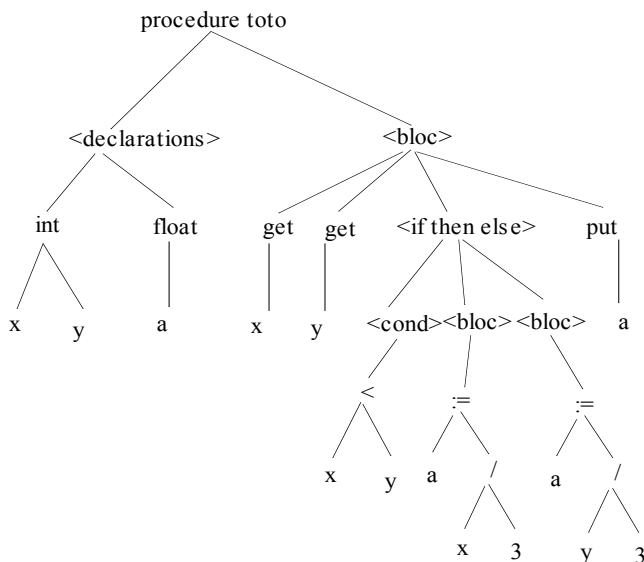
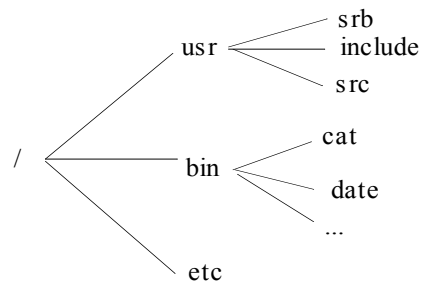


Analyse d'une expression arithmétique :

$(x+y) * (3z-t)$

75

Système de fichiers



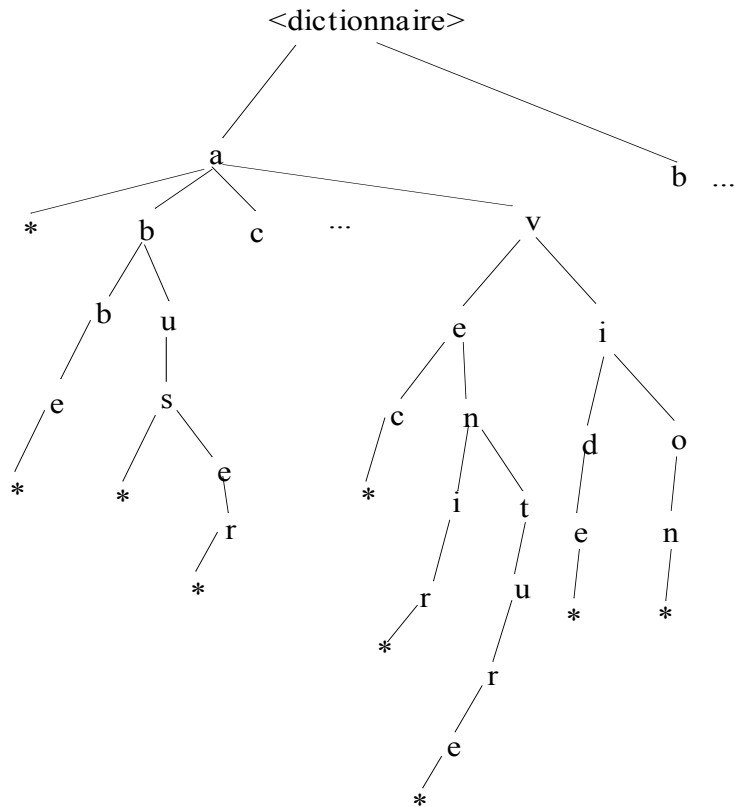
Analyse d'un programme (compilation):

```

procedure toto is
  x, y: integer;
  a : float;
begin
  get(x); get(y);
  if x<y then a:=float(x)/3.0;
  else a:=float(y)/3.0;
  endif;
  put(a);
end toto;
  
```

76

Dictionnaire de mots :



77

Spécification des arbres binaires *abin* (révision TAD !)

TA abin

UTILISE booléen, noeud

OPERATIONS

arbnouv : -> abin

enracine : noeud * abin * abin -> abin

ag, ad : abin -> abin

racine : abin -> noeud

est_vide : abin -> booléen

PRECONDITIONS avec un abin a

racine(a) ssi est_vide(a)=faux

ag(a) ssi est_vide(a)=faux

ad(a) ssi est_vide(a)=faux

AXIOMES avec un noeud e et deux abin a1 et a2

racine(enracine(e, a1, a2)) = e

ag(enracine(o, a1, a2)) = a1

ad(enracine(o, a1, a2)) = a2

...

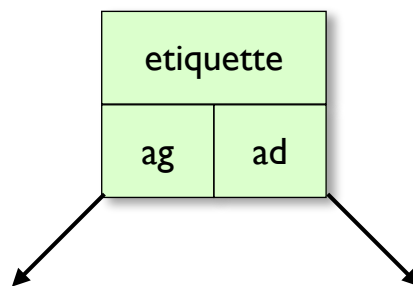
78

Structure de données

Les arbres binaires peuvent être naturellement représentés par des pointeurs :

```
type noeud = {  
  etiquette : entier  
  ag, ad    : pointeur noeud  
}  
type abin = pointeur noeud
```

Une forêt d'arbres binaires peut se représenter sous la forme d'une liste (chaînée ou contiguë) de `abin`.

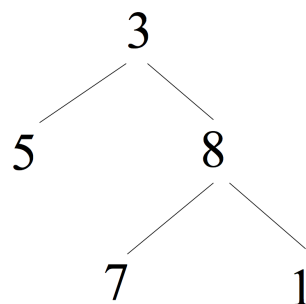


79

Exemple d'utilisation : construire un arbre

```
avec arb, a1, a2 : abin
```

```
a1 <- enracine(5, arbnouv, arbnouv)  
a2 <- enracine(8,  
               enracine(7, arbnouv, arbnouv),  
               enracine(1, arbnouv, arbnouv))  
arb <- enracine(3, a1, a2)
```

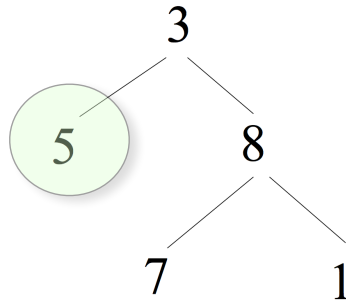


80

Exemple d'utilisation : construire un arbre

avec arb, a1, a2 : abin

```
a1 <- enracine(5, arbnouv, arbnouv)
a2 <- enracine(8,
               enracine(7, arbnouv, arbnouv),
               enracine(1, arbnouv, arbnouv))
arb <- enracine(3, a1, a2)
```

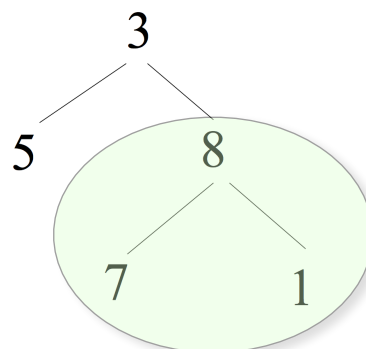


81

Exemple d'utilisation : construire un arbre

avec arb, a1, a2 : abin

```
a1 <- enracine(5, arbnouv, arbnouv)
a2 <- enracine(8,
               enracine(7, arbnouv, arbnouv),
               enracine(1, arbnouv, arbnouv))
arb <- enracine(3, a1, a2)
```

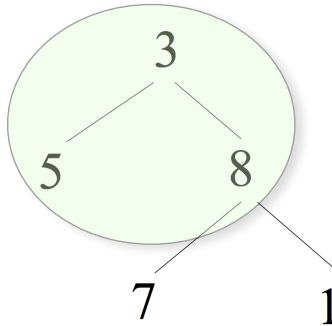


82

Exemple d'utilisation : construire un arbre

avec arb, a1, a2 : abin

```
a1 <- enracine(5, arbnouv, arbnouv)
a2 <- enracine(8,
               enracine(7, arbnouv, arbnouv),
               enracine(1, arbnouv, arbnouv))
arb <- enracine(3, a1, a2)
```



83

Hauteur d'un arbre

La **hauteur** (ou niveau ou profondeur) $h(x)$ d'un nœud x d'un arbre est définie par la relation de récurrence suivante :

$h(x) = 1$ si x est racine

$h(x) = 1 + h(y)$ avec y père de x .

La hauteur d'un arbre a non vide est donnée par:

$h(a) = \max\{ h(x), x \text{ nœud de } a \}$

Par définition, on considère que la hauteur d'un arbre vide est 0.

La longueur de cheminement d'un arbre a correspond à la somme de toutes les hauteurs de tous les nœuds de l'arbre. On parle aussi de cheminement interne et externe.

84

Hauteur d'un d'arbre

< à vous >

85

Hauteur d'un d'arbre

```
fonction hauteur(a : abin) retourne entier  
début
```

```
    avec    hg, hd : entier // hauteurs des sous arbres g/d
```

```
    si a = null alors retourne 0 fin si
```

```
    hg <- hauteur((contenu a).ag)
```

```
    hd <- hauteur((contenu a).ad)
```

```
    retourne 1 + max(hg, hd)
```

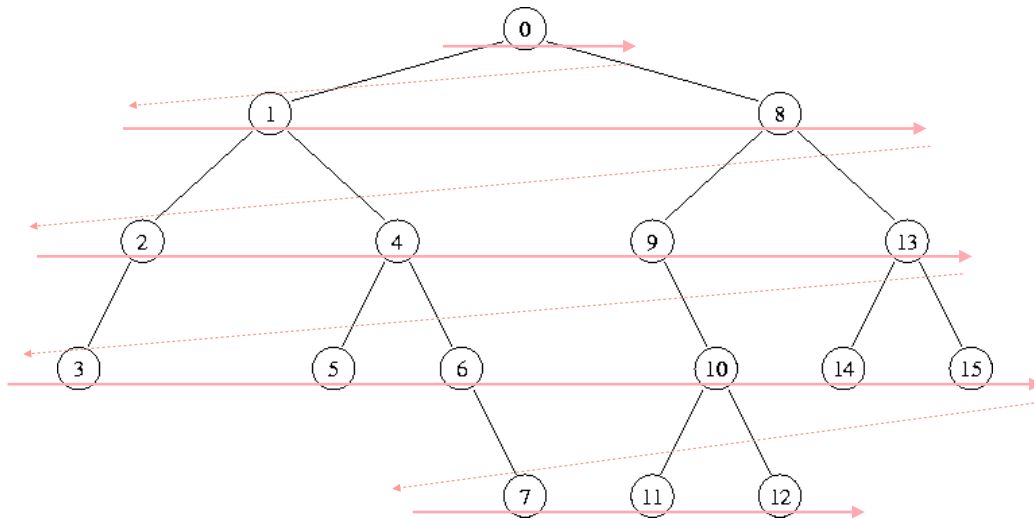
```
fin hauteur
```

86

Parcours d'arbre

En largeur :

On parcourt l'arbre niveau par niveau. Les nœuds de niveau 0 sont d'abord parcourus puis les nœuds de niveau 1 et ainsi de suite. Dans chaque niveau, les nœuds sont parcourus de la gauche vers la droite. Le parcours en largeur de l'arbre ci-dessous parcourt les nœuds dans l'ordre [0,1,8,2,4,9,13,3,5,6,10,14,15,7,11,12].

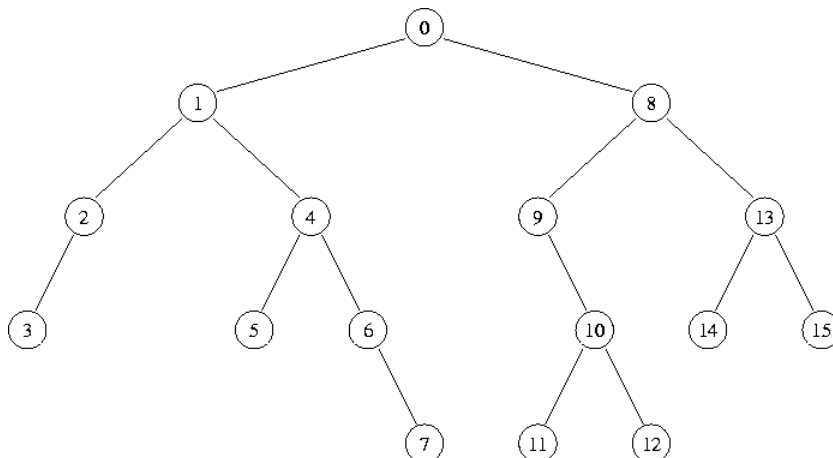


87

Les parcours en profondeur

Ils consistent à parcourir récursivement les sous-arbres gauche et droit de la racine. 3 parcours en profondeur peuvent être définis :

- *parcours préfixe* : la racine est traitée avant les appels récursifs sur les sous-arbres gauche et droit (faits dans cet ordre). [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15].
- *parcours infixe* : le traitement de la racine est fait entre les appels sur les sous-arbres gauche et droit. [3,2,1,5,4,6,7,0,9,11,10,12,8,14,13,15].
- *parcours suffixe*, la racine est traitée après les appels récursifs sur les sous-arbres gauche et droit (faits dans cet ordre). [3,2,5,7,6,4,1,11,12,10,9,14,15,13,8,0].



88